

Skip List – Insert Element

–Program Analysis

–Insertion(3)[Insert In Between]

SkipList * list

Address **list**

ff33 *ff08*

Address **level** *** header**

ff08 2 *ff12*

Address **key** *** forward**

ff12 -1 *ff80*

Address

Values

<i>forward[0]</i>	<i>ff80</i>	<i>ff16</i>
<i>forward[1]</i>	<i>ff88</i>	<i>ff16</i>
<i>forward[2]</i>	<i>ff90</i>	<i>ff16</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>

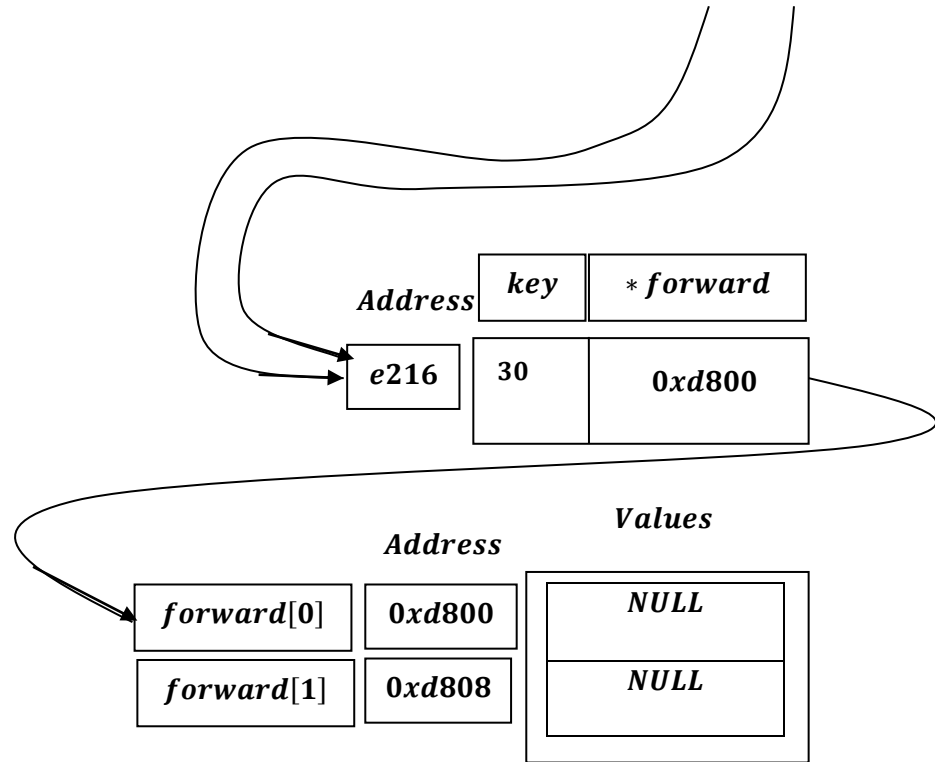
Address **key** *** forward**

ff16 20 *0xb400*

Address

Values

<i>forward[0]</i>	<i>0xb400</i>	<i>e216</i>
<i>forward[1]</i>	<i>0xb408</i>	<i>e216</i>
<i>forward[2]</i>	<i>0xb410</i>	<i>NULL</i>



```
case 1:
    cout << "Enter key to insert: ";
    cin >> key;
    insertElement(list, key);
    break;
```

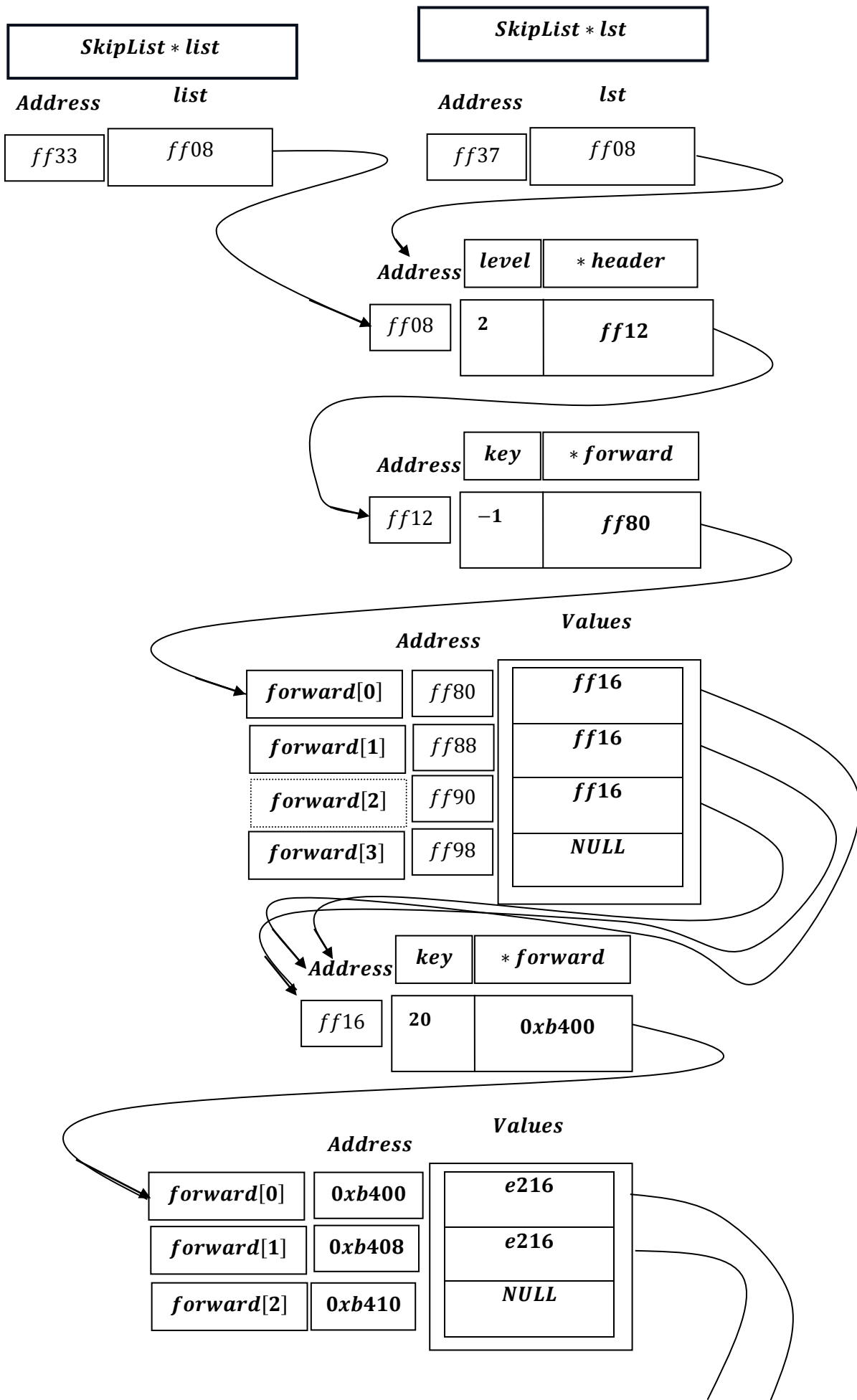
Entering : Key = 25 .

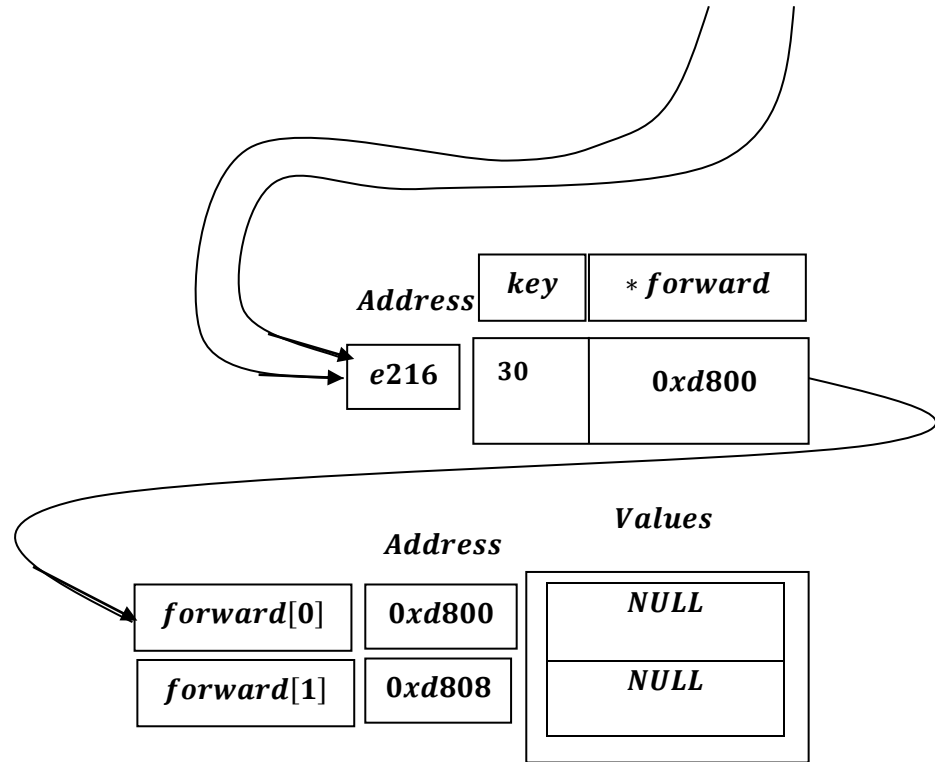
insertElement(list: ff08 ,key: 25);

Now ,we have :

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 25.*





```
struct Node *current = lst->header;
```

current = lst → header : ff12.

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

<i>Array Element</i>	<i>Hypothetical Stack Address</i>	<i>Value (Garbage Data)</i>	<i>Pointer Status</i>
<i>update[0]</i>	<i>0x7fff0000 to 0x7fff0007</i>	<i>0x4a3f8921bce45099</i>	<i>Wild Pointer (Points to a random, invalid address)</i>
<i>update[1]</i>	<i>0x7fff0008 to 0x7fff000f</i>	<i>0x0000000000000002</i>	<i>Wild Pointer</i>
<i>update[2]</i>	<i>0x7fff0010 to 0x7fff0017</i>	<i>0x7ffee1245a00881c</i>	<i>Wild Pointer</i>
<i>update[3]</i>	<i>0x7fff0018 to 0x7fff001f</i>	<i>0x0000000000000000</i>	<i>Accidentally NULL (Just by chance)</i>

More simpler way:

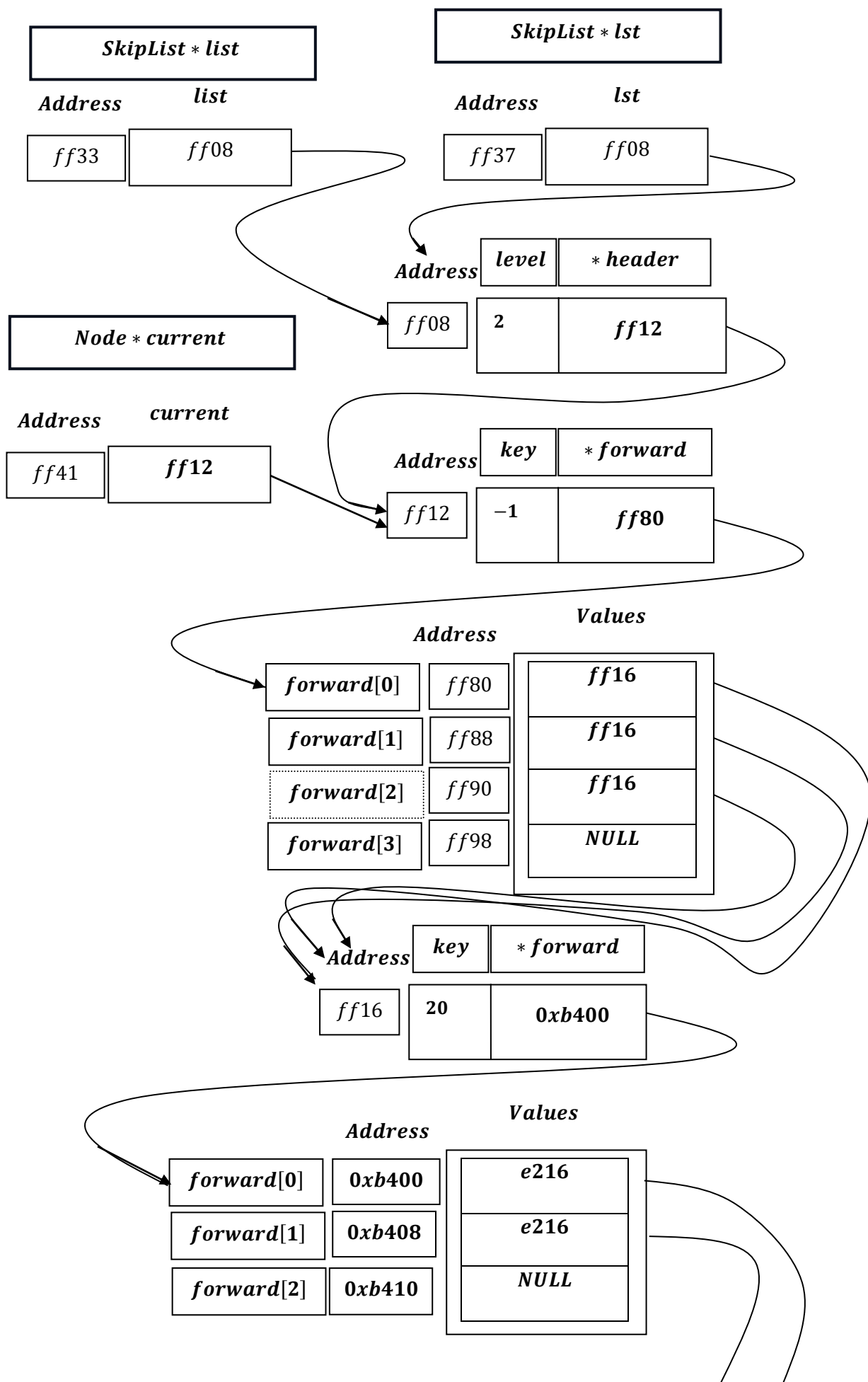
	<i>Address</i>	<i>Values</i>	
<i>update[0]</i>	<i>0x7fff0000</i>	<i>0x4a3f8921bce45099</i>	<i>[8 bytes]</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>0x0000000000000002</i>	<i>[8 bytes]</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>0x7ffee1245a00881c</i>	<i>[8 bytes]</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>0x0000000000000000</i>	<i>[8 bytes]</i>

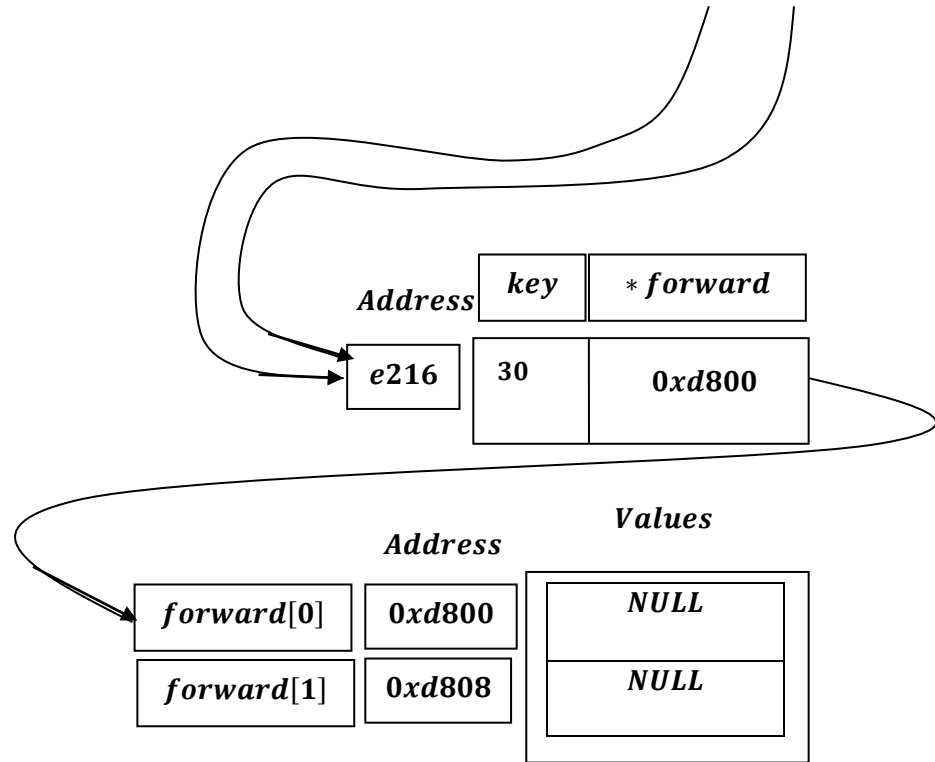
```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

``memset`` stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	<i>0x7fff0000</i>	<i>NULL</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>NULL</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>NULL</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>NULL</i>

*Here we kept addresses same like: 0x7fff0000
, just for an example , but it may not be same ,
in reality.*





	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

For : $i = lst \rightarrow level = 2 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current \rightarrow forward[2] = ff16 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

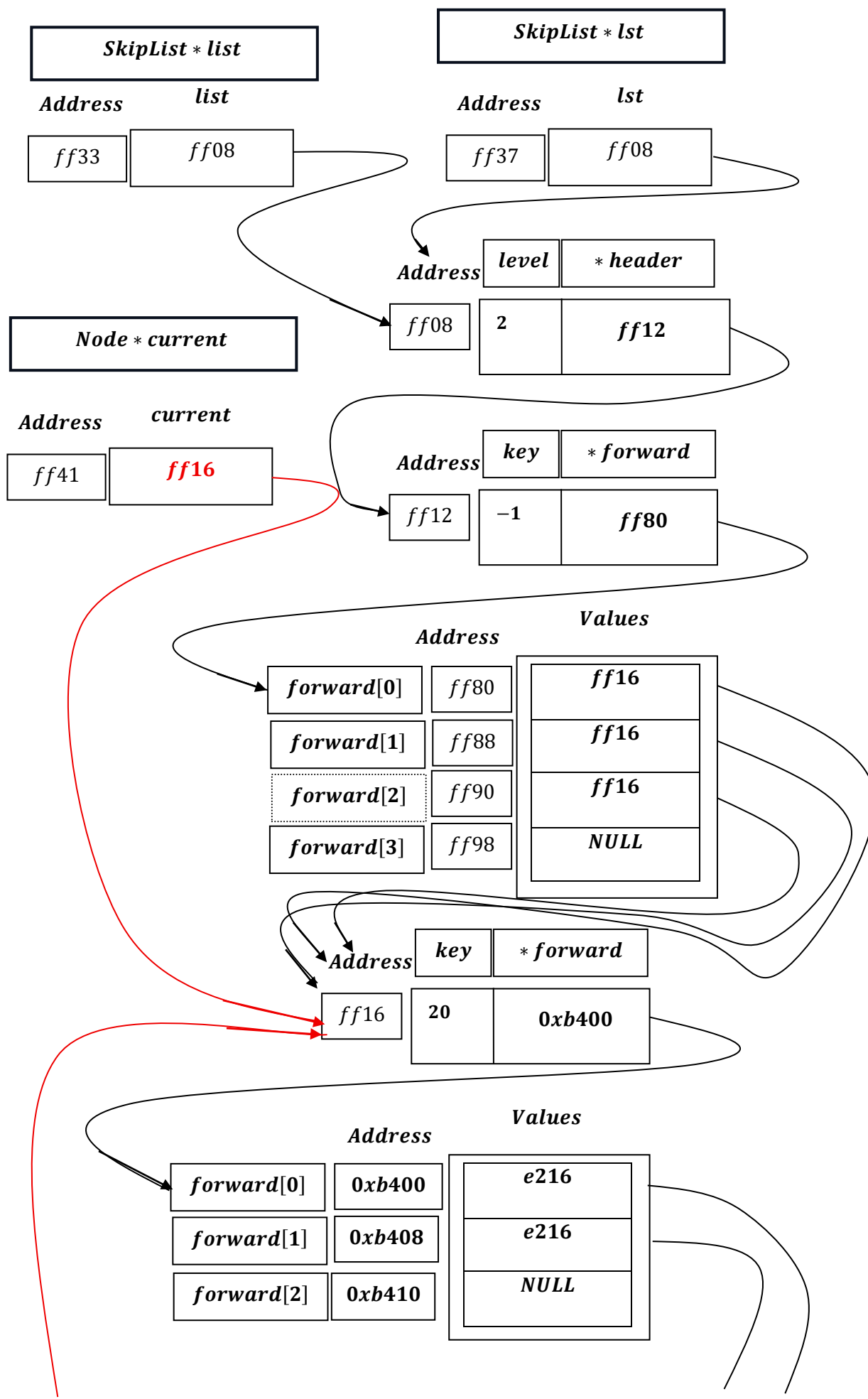
$current \rightarrow forward[2] \rightarrow key: 20 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

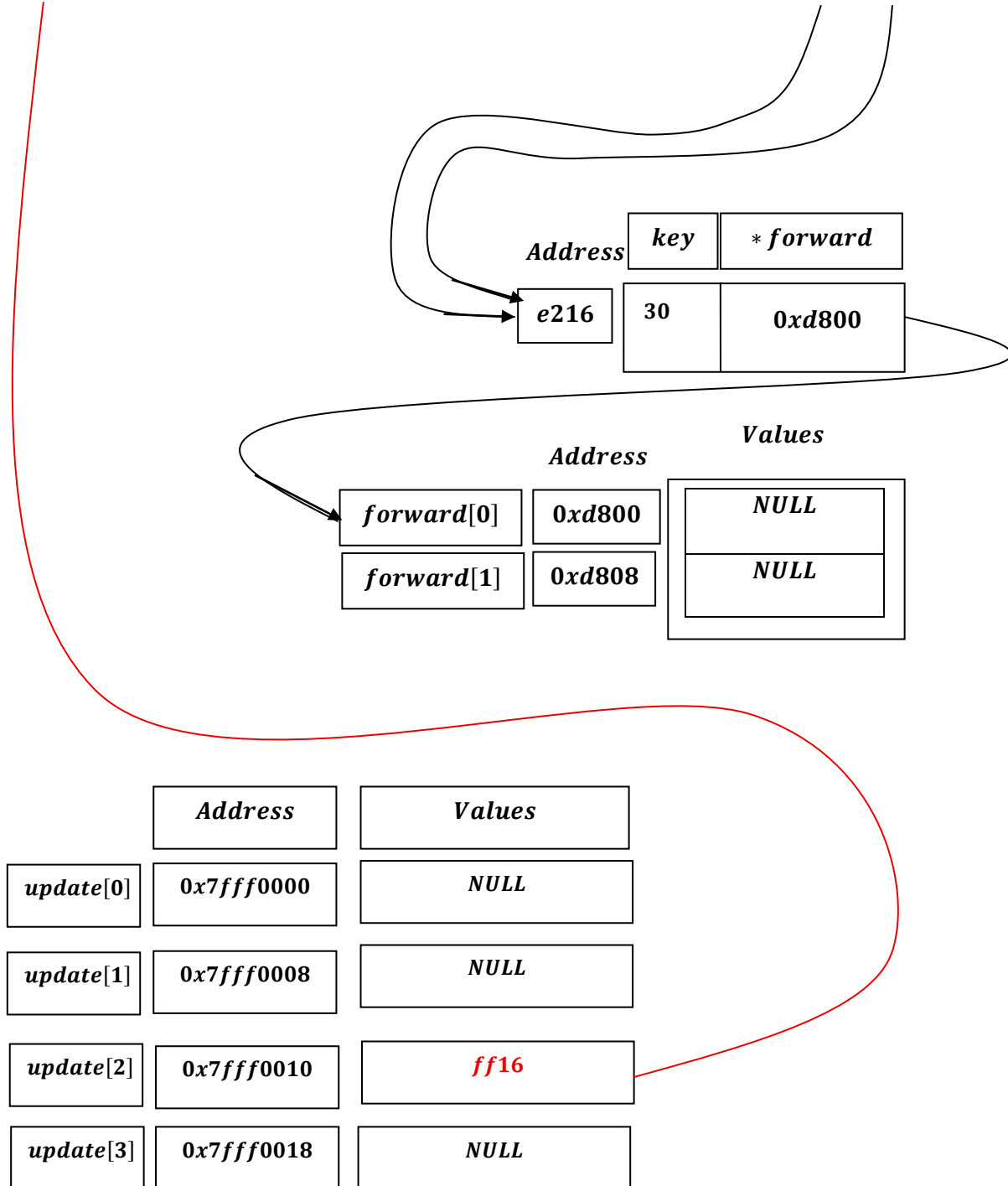
$current = current \rightarrow forward[2] = ff16$

while: $current \rightarrow forward[2] = NULL \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False, while} \\ \text{loop exits} \end{array} \right]$

$update[2] = current = ff16.$

$i = i - 1 = 2 - 1 = 1.$





For : $i = 1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

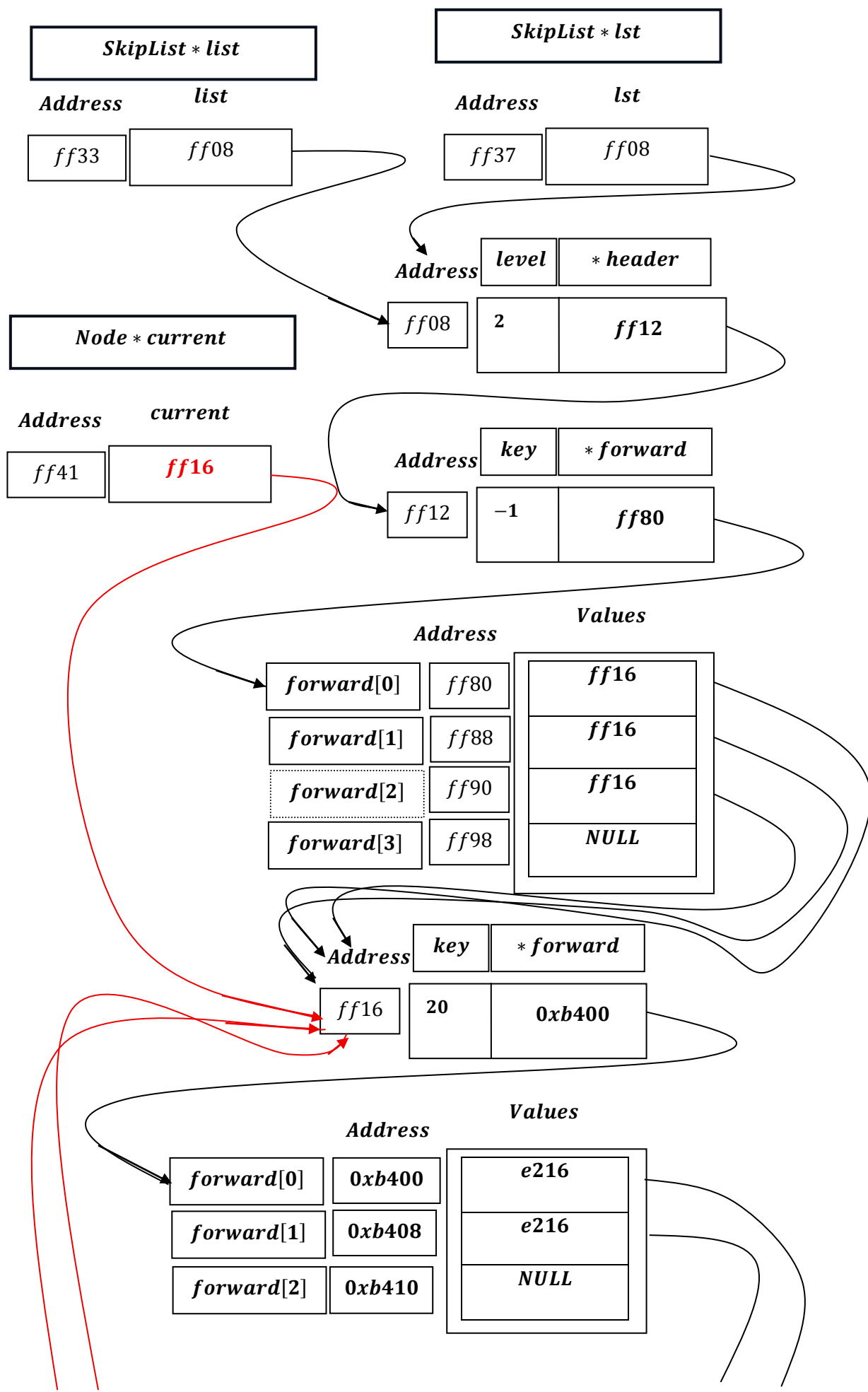
while: $current \rightarrow forward[1] = e216 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is true} \end{array} \right]$ And

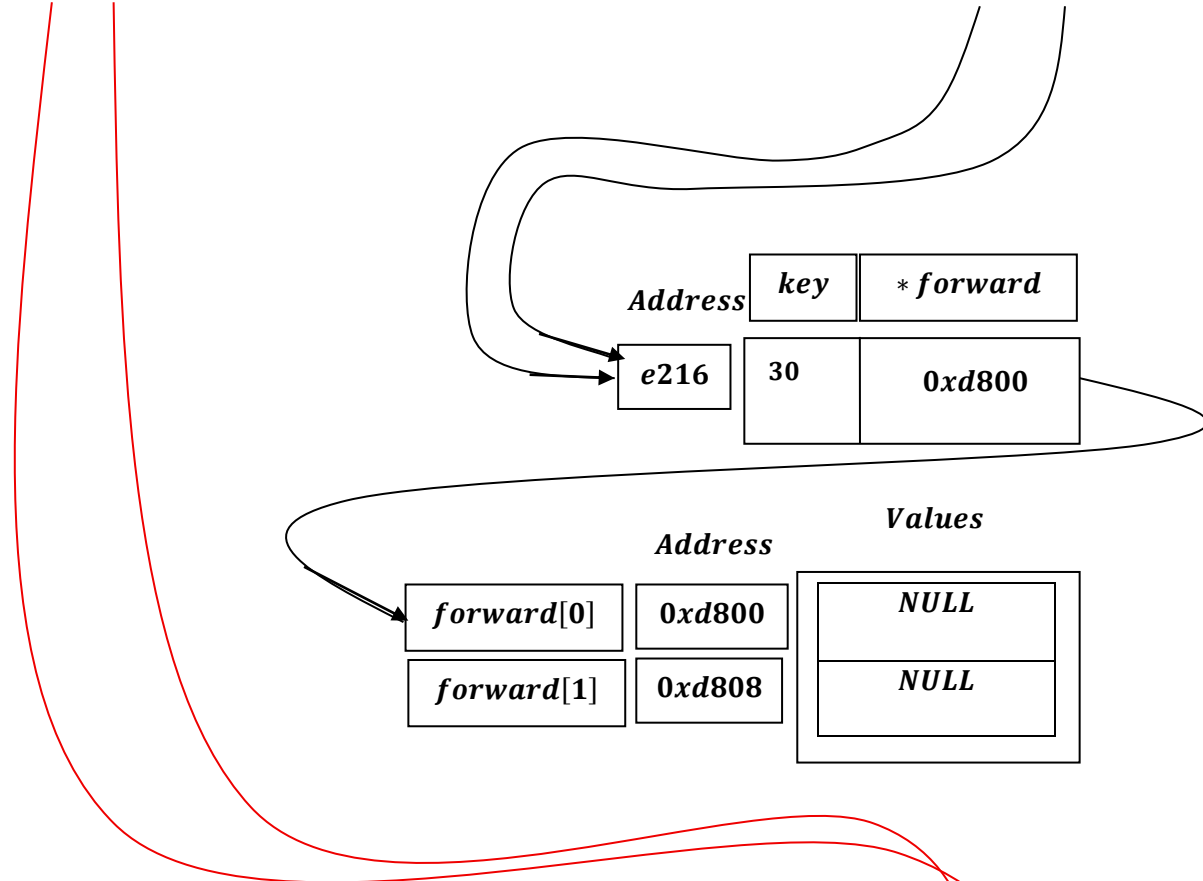
$current \rightarrow forward[1] \rightarrow key: 30 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[hence, while loop exits]

$update[1] = current = ff16$.

$i = i - 1 = 1 - 1 = 0$.





	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	ff16
update[2]	0x7fff0010	ff16
update[3]	0x7fff0018	NULL

For : $i = 0 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

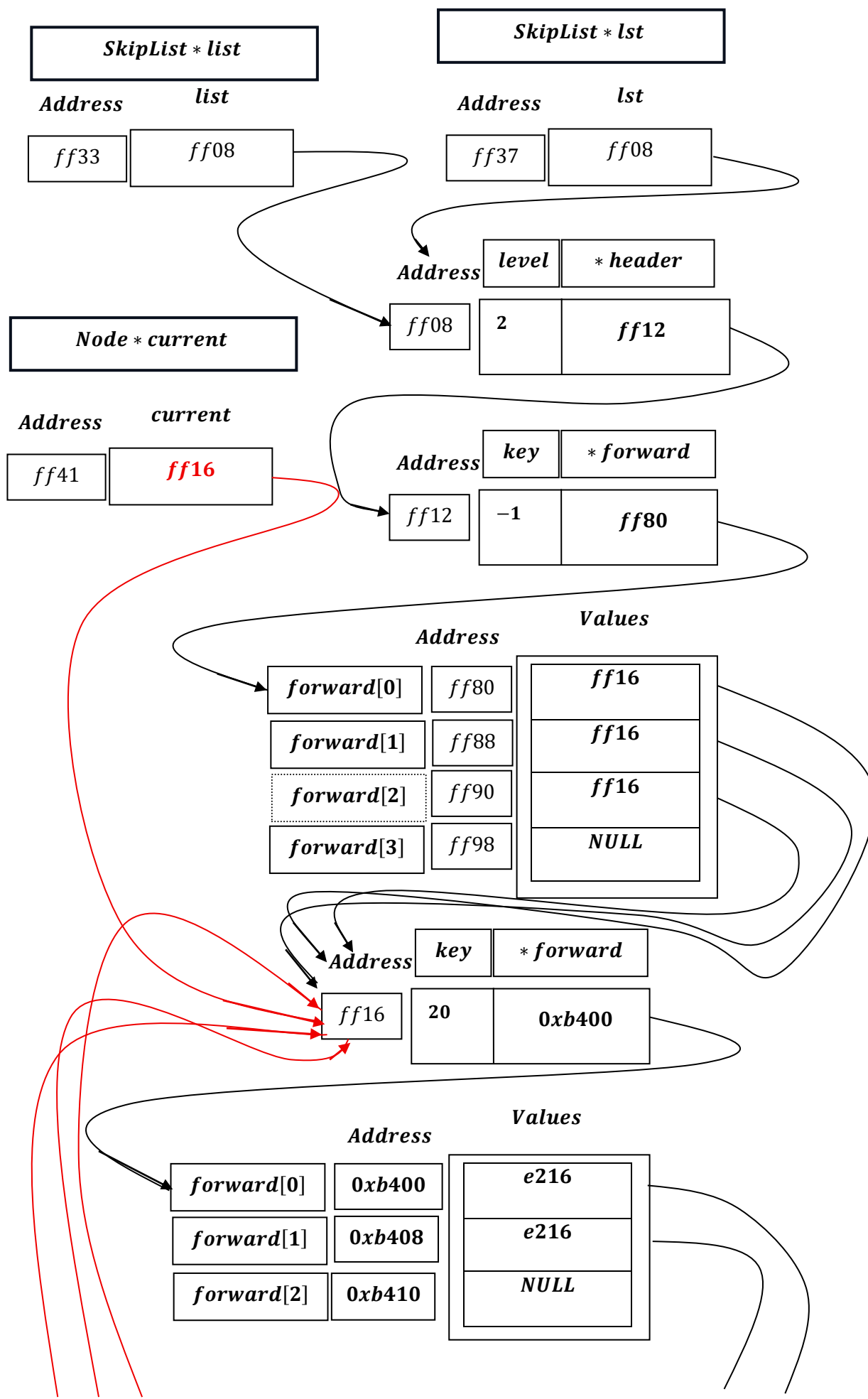
while: $\text{current} \rightarrow \text{forward}[0] = \text{e216} \neq \text{NULL}$ $\left[\begin{array}{l} \text{Condition} \\ \text{is true} \end{array} \right]$ And

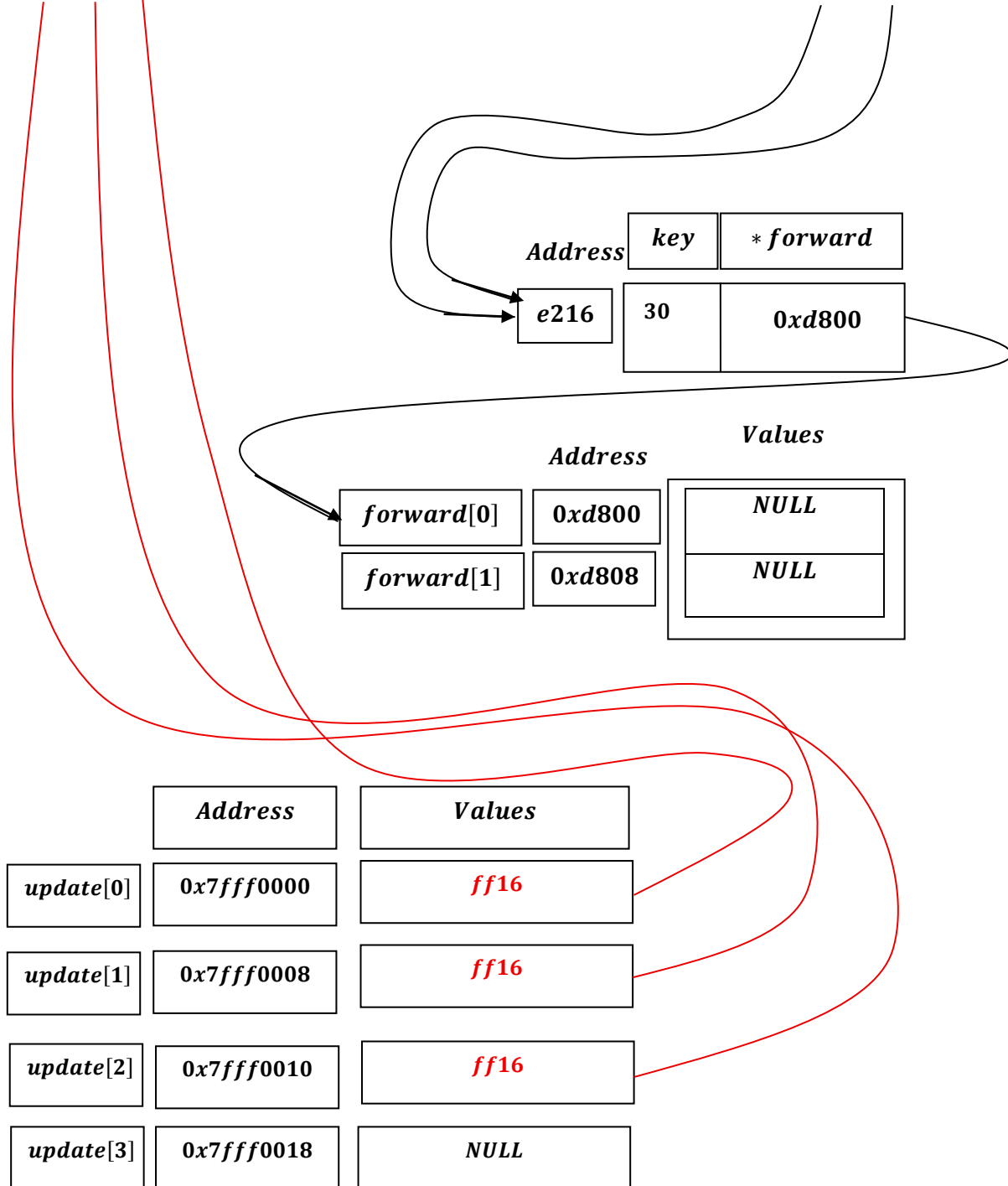
$\text{current} \rightarrow \text{forward}[0] \rightarrow \text{key: } 30 < \text{key: } 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[hence, while loop exits]

$\text{update}[0] = \text{current} = \text{ff16}.$

$i = i - 1 = 0 - 1 = -1.$

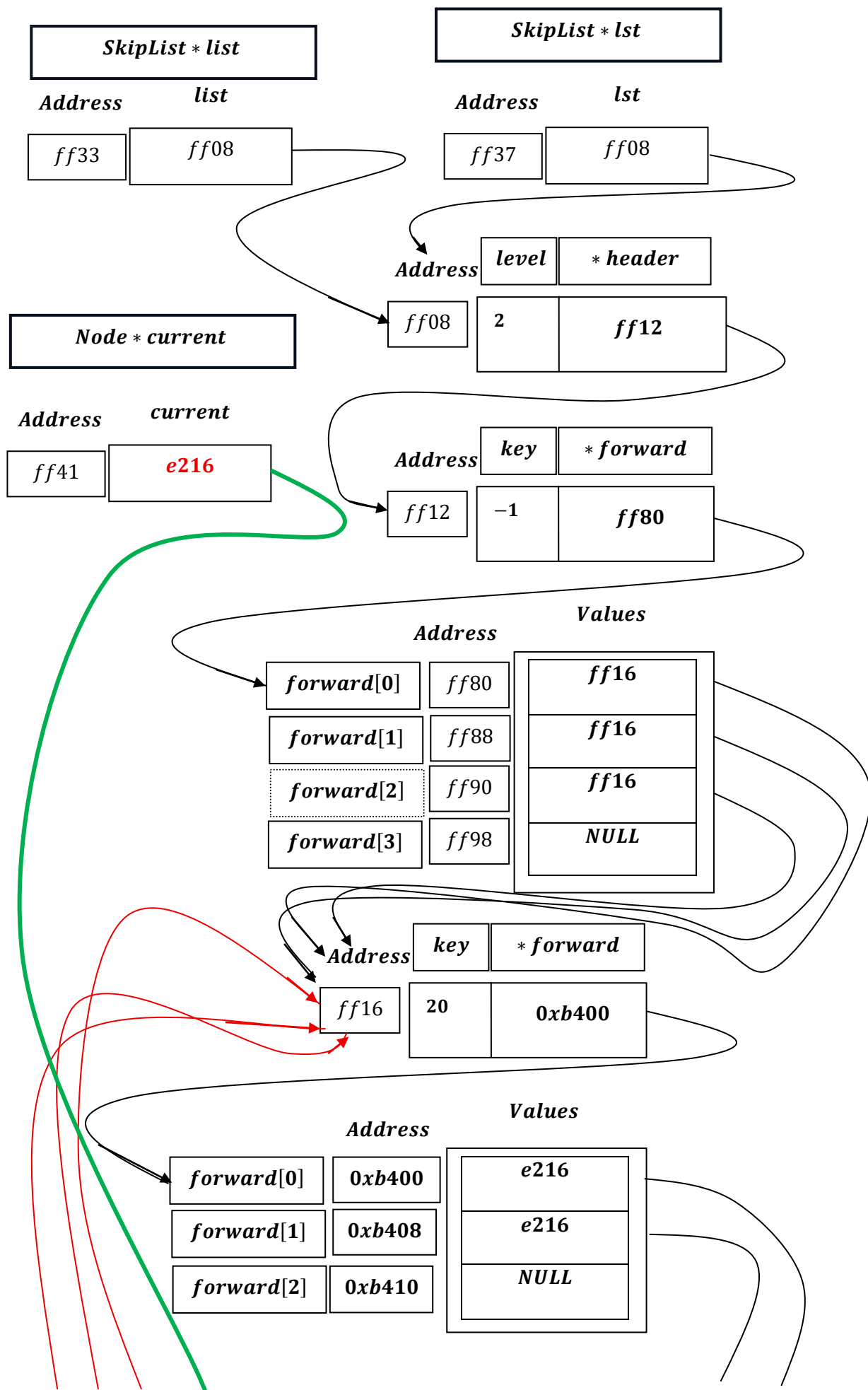


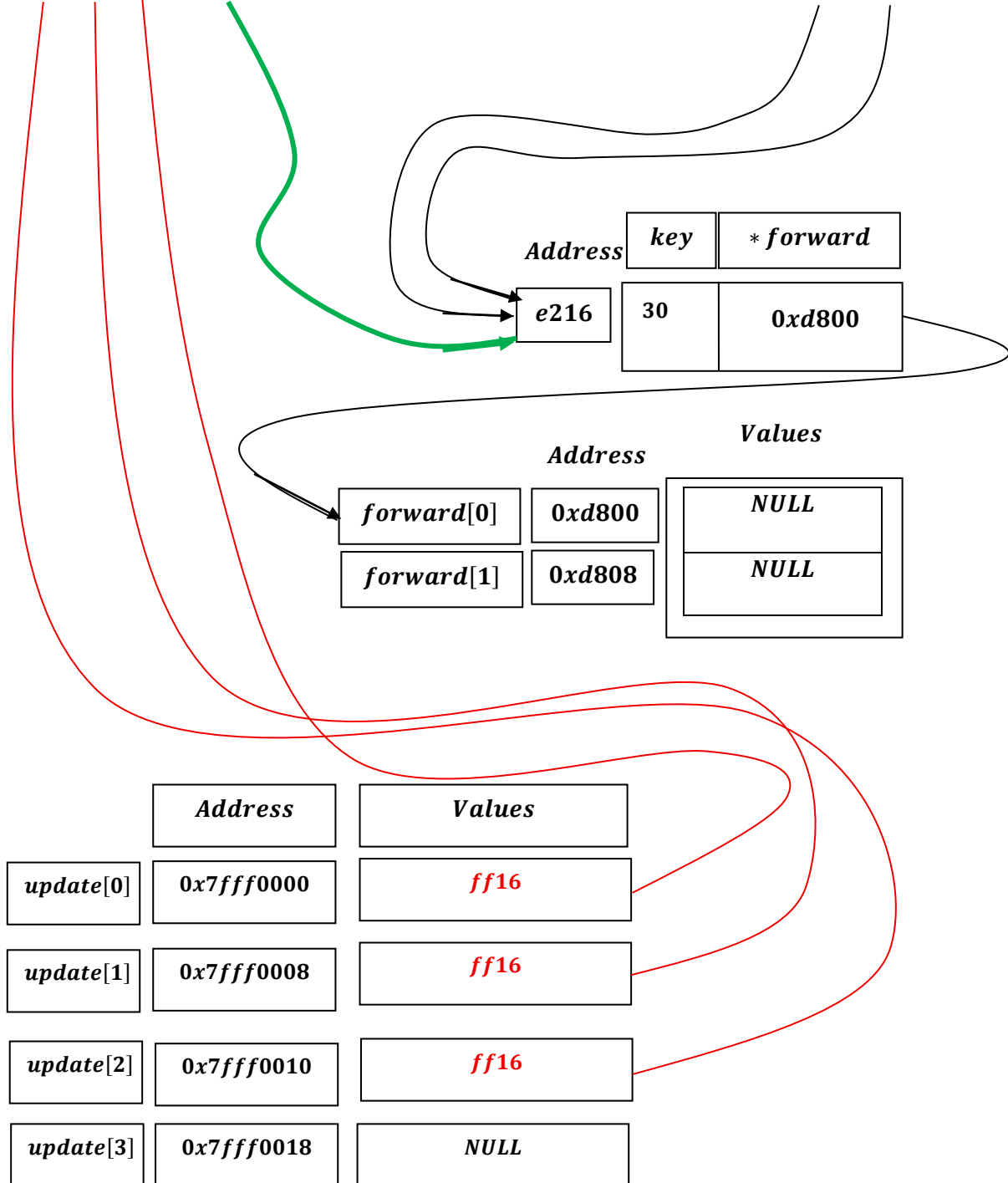


For : $i = -1 ; i \geq 0$ $\left[\begin{matrix} \text{Condition} \\ \text{is False} \end{matrix} \right]$, condition is false and loop exits.

```
current = current->forward[0];
```

current = current → forward[0] = e216.





```
if (current == NULL || current->key != key) {...}
```

current = *e216* $\neq$ *NULL* (*false*) OR *current* $\rightarrow$ *key* = *30* $\neq$ *key* = *25* (*True*),
 hence if block executes.

```
int rlevel = randomLevel();
```

rlevel = *randomlevel*() = *1* (say)

```

if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}

```

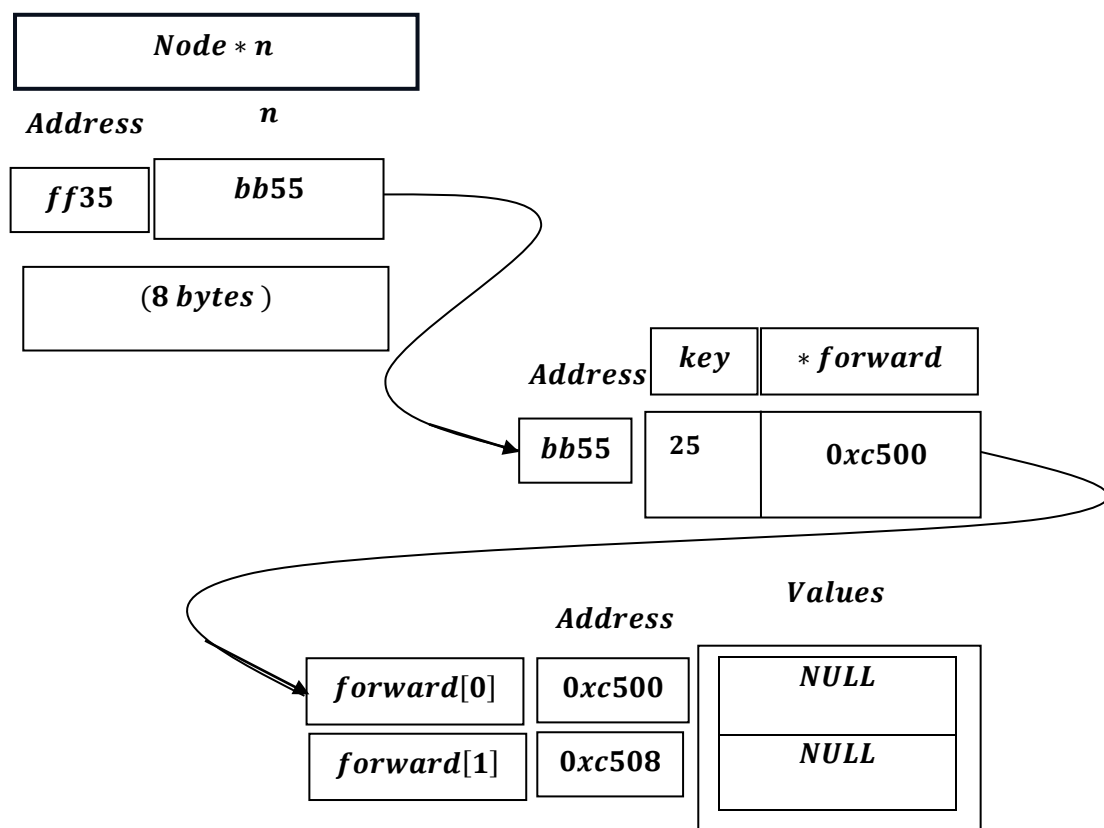
if $rlevel = 1 > lst \rightarrow level = 2$ is false $\left[\begin{matrix} \text{Condition is} \\ \text{false} \end{matrix} \right]$, if block exits.

```

struct Node *n = createNode(key, rlevel){...};

```

Let $rlevel = 1$ and $key = 25$.



```

return n;

```

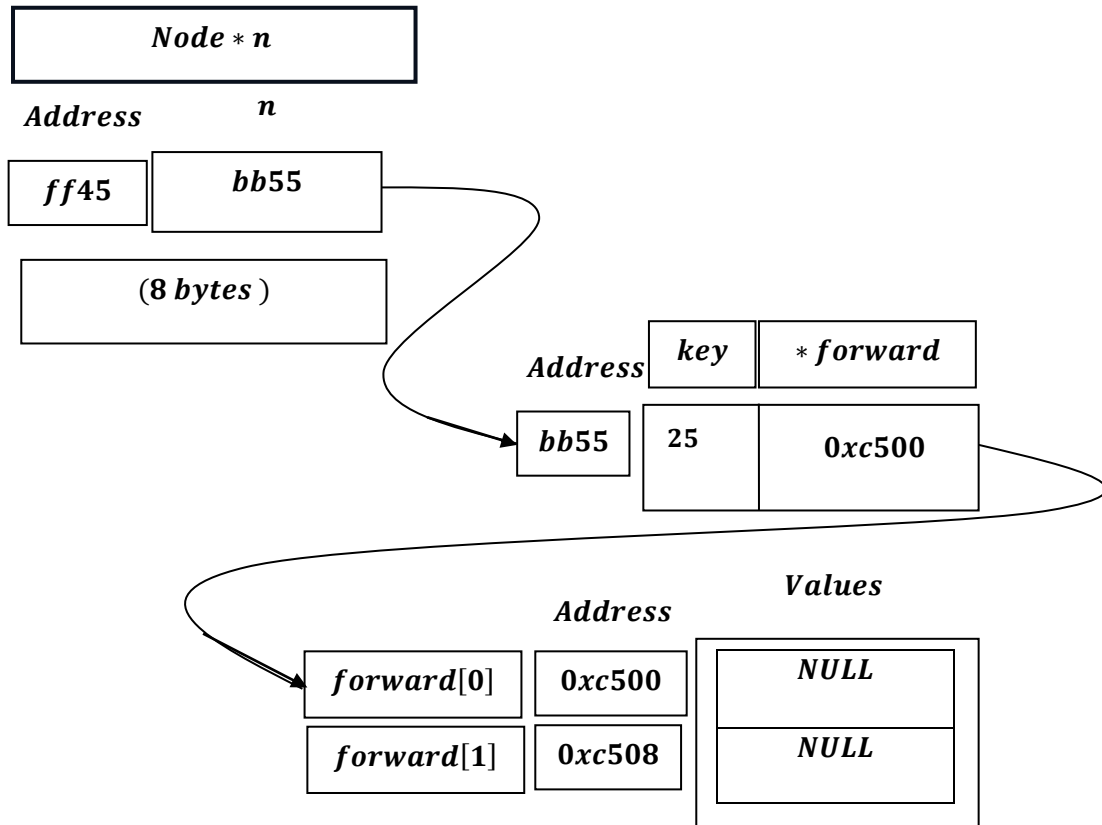
i. e. Return $n = bb55$

Actual structure

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
<i>bb55</i>	<i>key = 25</i>	<i>key[bb55 – bb58] 4 bytes for int</i>
		<i>padding → [bb59 – bb5c] 4 bytes for padding</i>
<i>bb5d</i>	<i>Address: 0xc500</i>	<i>* forward → [bb5d – bb64] 8 bytes for pointer variable</i>
<i>0xc500</i>	<i>NULL</i>	<i>forward[0] → [0xc500 – 0xc507] 8 bytes for pointer variable</i>
<i>0xc508</i>	<i>NULL</i>	<i>forward[1] → [0xc508 – 0xc50f] 8 bytes for pointer variable</i>

struct Node * n = createNode(key = 25, rlevel = 1)

struct Node * n = bb55



```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

int i = 0; i ≤ rlevel = 1

n → forward[0] = update[0] → forward[0] = e216

update[0] → forward[0] = n = bb55

i = i + 1 = 0 + 1 = 1

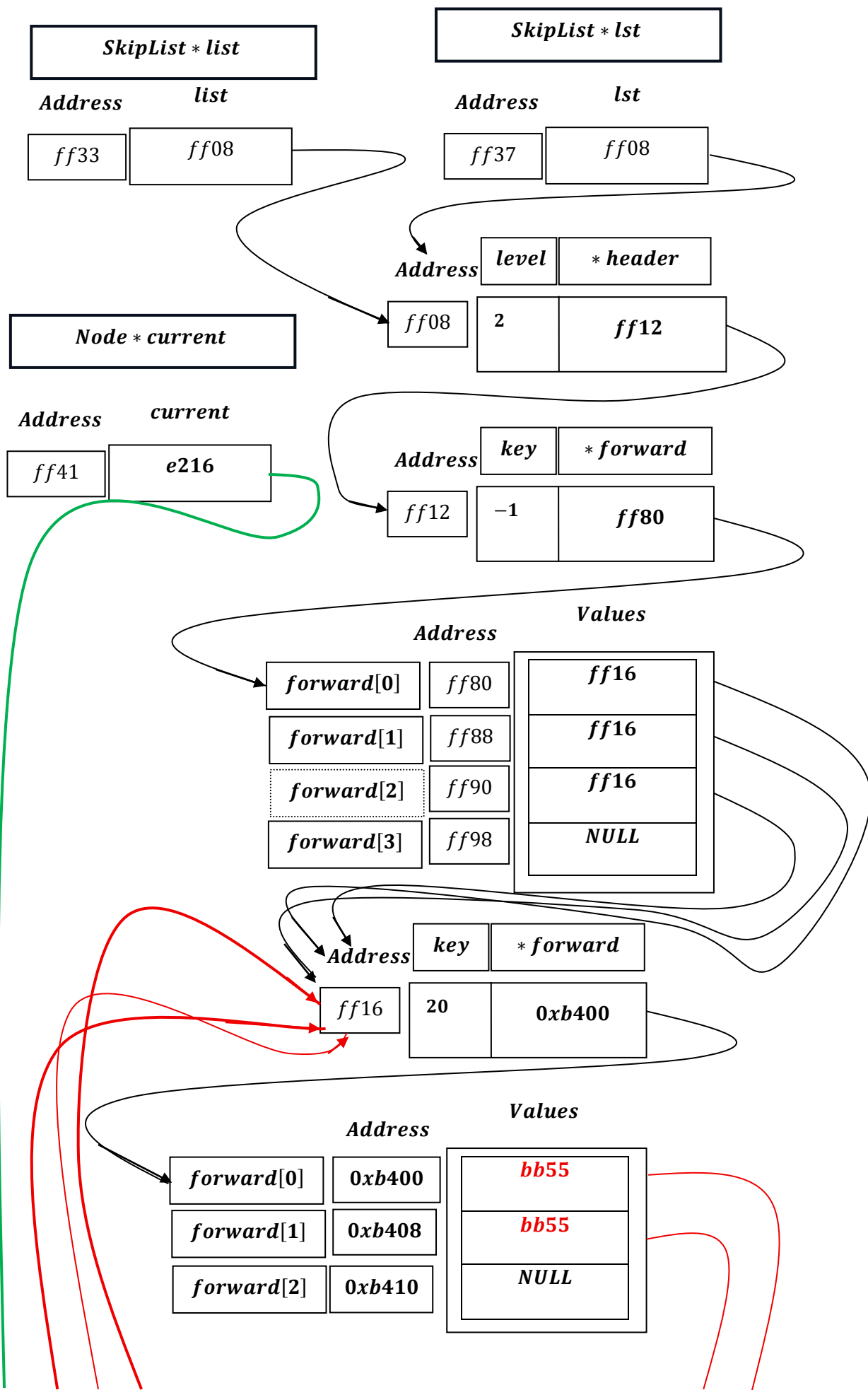
int i = 1; i ≤ rlevel = 1

n → forward[1] = update[1] → forward[1] = e216

update[1] → forward[1] = n = bb55

i = i + 1 = 1 + 1 = 2

int i = 2; i ≤ rlevel = 1 $\left[\begin{array}{l} \textit{Condition} \\ \textit{is False} \end{array} \right]$



*Node * n*

Address

n

ff45

bb55

Address

key

** forward*

(8 bytes)

bb55

25

0xc500

Address

Values

forward[0]

0xc500

e216

forward[1]

0xc508

e216

Address

key

** forward*

e216

30

0xd800

Address

Values

forward[0]

0xd800

NULL

forward[1]

0xd808

NULL

Address

Values

update[0]

0x7ff0000

ff16

update[1]

0x7ff0008

ff16

update[2]

0x7ff0010

ff16

update[3]

0x7ff0018

NULL

```
cout << "Successfully inserted key " << key << endl;
```

Successfully inserted key : 25 and function ends .

As , the function ends :

*SkipList * lst*

Address	lst
ff37	ff08

*Node * current*

Address	current
ff41	e216

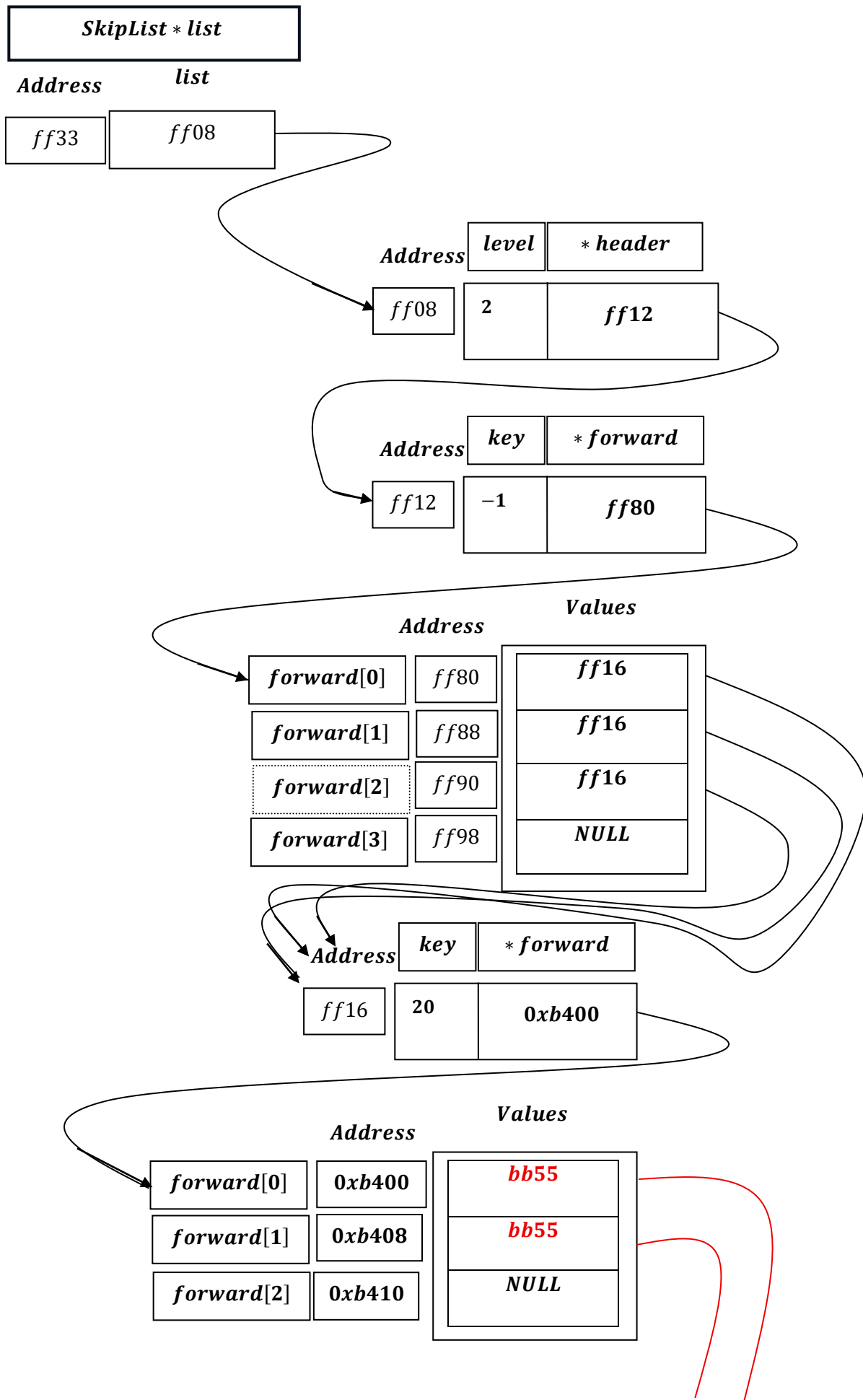
	Address	Values
update[0]	0x7fff0000	ff16
update[1]	0x7fff0008	ff16
update[2]	0x7fff0010	ff16
update[3]	0x7fff0018	NULL

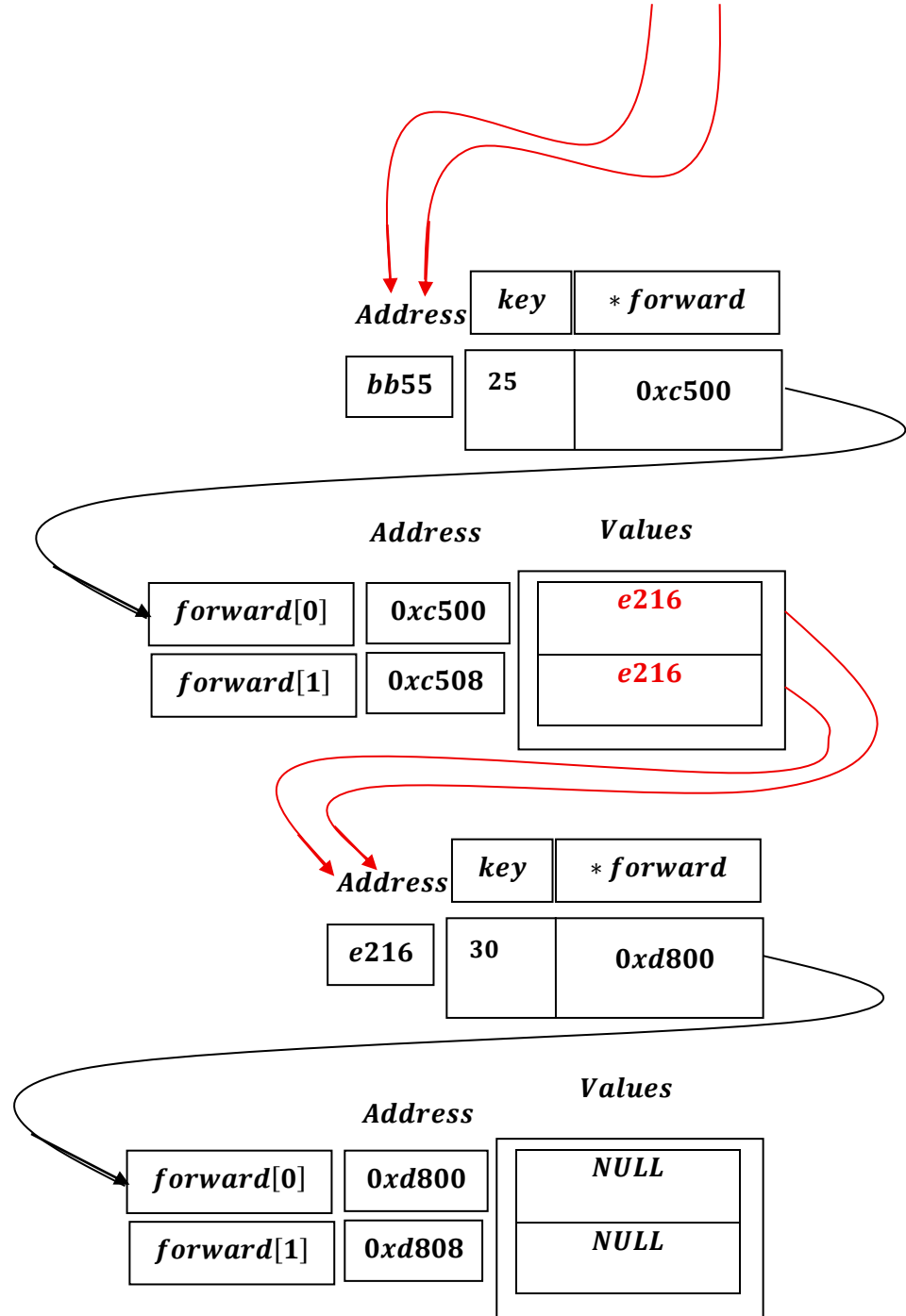
*Node * n*

Address	n
ff45	bb55

*Gets
Destroyed*

And we are left with:





But ,what happens,when ,we increase rand() value i.e. 3 > lst → level: 2, then what happens? Lets discuss this on next